

SalesGodCRM calling verification

SalesGodCRM browser calling verification

Run started 2026-10-06T15:43:02Z, finished 2026-10-06T15:46:37Z. Harness code: harness/. Evidence: evidence/.

Run outcome

No live calls were placed in this run. The preflight checks failed, so every behavioral test fell back to documentary evidence or was marked BLOCKED. No status in this report was upgraded without evidence; nothing below is CONFIRMED or REFUTED, because those statuses require observed behavior.

Preflight check	Result	Detail
.env inputs SW_SPACE, SW_PROJECT_ID, SW_API_TOKEN	FAIL	missing: ['SW_SPACE', 'SW_PROJECT_ID', 'SW_API_TOKEN']
reach https://signalwire.com/api/relay/rest/phone_numbers	FAIL	ProxyError('403 Forbidden')
reach https://signalwire.com/docs/llms.txt	FAIL	ProxyError('403 Forbidden')
reach https://api.trycloudflare.com	FAIL	ProxyError('403 Forbidden')

To run live: place .env (SW_SPACE, SW_PROJECT_ID, SW_API_TOKEN, optional SW_SIGNING_KEY for B11) in verification/, allow outbound HTTPS to the Space host and signalwire.com, and allow a public tunnel (cloudflared quick tunnel needs api.trycloudflare.com plus outbound connections to Cloudflare's edge on port 7844; or provide NGROK_AUTHTOKEN). Then run python -m harness.run. The same code path produces this report with live statuses.

Status counts: BLOCKED: 14, DOCUMENTED (not observed): 16, NOT DETERMINABLE EMPIRICALLY: 4

Local self-tests (no Space required)

These prove the harness's own components, not SignalWire behavior.

Self-test	Result	Output
selftest_audio	pass	{"fixtures_exist": true, "ch0_is_pstn": true, "ch1_is_browser": true, "silence_gap_found": true, "skip_shorter": true, "skip_no_gap": true}
selftest_signature	pass	{"sha1_matches_official_build_signature": true, "official_validates_our_sig": true, "our_validator_accepts": true, "sha256_accepted": true, "tampered_body_rejected": true, "wrong_key_rejected": true, "form_matches_official_compat": true}
selftest_browser	pass	{"umd_global_present": true, "SignalWire_class": true, "StaticCredentialProvider": true, "TokenRefreshError_exported": true, "page_reported_to_backend": true, "register_without_token_fails_cleanly": true}
selftest_mock	pass	{"a1_confirmed_with_ringing_devices": true, "a4_captured_call_fields": true, "anchor_id_extracted": true, "hangup_sent_to_anchor": true, "denylist_blocks_911": true, "unowned_number_blocked": true, "a1_refuted_when_no_ringing": true}

Executive summary

ID	Question	Status	One-line answer
A1	Does calling one Subscriber ring all of its registered devices?	DOCUMENTED (not observed)	Documented yes: dialing a Subscriber rings every live registration at once; not yet observed.

ID	Question	Status	One-line answer
A2	First-answer-wins: when one device answers, do the others stop ringing, and how fast?	BLOCKED	Unknown: no document states what the other devices see or how fast; live test required.
A3	Does reject() on one device stop all devices on that Subscriber, and other Subscribers in the same parallel group?	BLOCKED	Unknown: reject() stops the rejecting device, but its effect on other devices or parallel Subscribers is undocumented.
A4	What does the inbound SWML webhook request contain?	DOCUMENTED (not observed)	POST JSON with call, vars, envs, params; call has call_id, from, to, type, direction, headers and more.
A5	Can the browser dial an SWML webhook address with userVariables, and what caller identity does the webhook receive?	DOCUMENTED (not observed)	dial(address, { userVariables }) is the confirmed v4 shape; the webhook field for userVariables and caller identity is undocumented.
A6	connect serial fallback to a phone number after the Subscriber times out, and connect_result branching	DOCUMENTED (not observed)	connect.serial gives fallback; connect_result is connected or failed and drives the voicemail branch.
A7	Exact REST command names and payloads for record start, pause, resume, stop, and hangup	DOCUMENTED (not observed)	calling.record, calling.record.pause (behavior skip or silence), calling.record.resume, calling.record.stop, calling.end.
A8	Stereo MP3 of both parties: format and channel mapping, inbound and outbound	BLOCKED	Unknown: stereo MP3 is supported, but channel-to-party mapping is undocumented; analyzer ready.
A9	Which status webhooks fire for connect, recording and call end, and where is the REST recording callback configured?	DOCUMENTED (not observed)	calling.call.connect, calling.call.state and calling.call.record events; REST recording callback goes in params.status_url.
A10	Which call_id must REST commands target (anchor rule)?	BLOCKED	Unknown: no anchor-leg rule is documented; live test sends commands to both legs.
A11	Maximum devices per Subscriber	BLOCKED	Unknown: no device limit is documented in the docs or the SDK.
A12	Cost per call by product	BLOCKED	No per-call cost data (no calls placed); list rates WebRTC \$0.003/min, local PSTN \$0.0066 in / \$0.008 out, recording \$0.002/min.
B1	No device registered for a Subscriber in the parallel list: fail fast or wait for timeout?	BLOCKED	Unknown: fail-fast versus timeout with zero registrations is undocumented.
B2	Parallel destination cap for connect.parallel	BLOCKED	Unknown: no parallel cap is documented; the 20-destination figure appears in no source.
B3	Does an expiring SAT drop an active call?	BLOCKED	Unknown: nothing says an established call drops at SAT expiry.
B4	Does automatic refresh work (backend refresh vs fingerprint + sat:refresh)?	DOCUMENTED (not observed)	Documented: refresh() is called 5 s before expiry with 5 retries, then TokenRefreshError; sat:refresh enables SDK self-refresh.
B5	Does deleting a Subscriber revoke active sessions and in-progress calls?	BLOCKED	Unknown: delete returns 204; effect on live sessions and calls is undocumented.
B6	Can custom data reach the browser on inbound calls?	DOCUMENTED (not observed)	Only userVariables is a documented path; the v4 Call object has no headers property.
B7	What caller ID does the browser see on inbound, and can connect.from override it?	BLOCKED	Unknown for PSTN callers: from maps to the invite caller_id_number; override behavior undocumented.
B8	What does the remote party hear during browser hold, and is it recorded?	BLOCKED	Unknown: v4 offers toggleHold(); what the remote party hears is undocumented.
B9	Can a device on an active call receive a second inbound call?	DOCUMENTED (not observed)	SDK lists every inbound call, including a second one; platform offer behavior unverified.
B10	Server-side transfer of a connected call to another Subscriber and to a phone number	DOCUMENTED (not observed)	calling.transfer with dest pointing at SWML that connects to the target is the documented method.
B11	Webhook authenticity: signature header and validation	DOCUMENTED (not observed)	X-SignalWire-Signature = hex HMAC-SHA1(Signing Key, url + raw_body); validator proven against SignalWire's library.
B12	Webhook reliability: retries and timeouts when the backend is slow or returns 500	DOCUMENTED (not observed)	About 5 s wait, then fallback_request_url on timeout or 5xx; SWML retry counts are undocumented.
B13	Recording retention and deletion via API	DOCUMENTED (not observed)	List, get and delete via /api/relay/rest/recordings; retention is indefinite until deleted.
B14	Outbound CPS: what happens when the default limit is exceeded?	DOCUMENTED (not observed)	Default 1 CPS; excess calls are queued (10,000 backlog), not rejected.
B15	Negotiated codecs on the WebRTC leg and the PSTN leg	BLOCKED	Unknown: read via call.rtcPeerConnection.getStats(); no codec getter in the SDK.
B16	Can one Subscriber hold multiple simultaneous connected calls across devices?	BLOCKED	Unknown: concurrent calls per Subscriber are undocumented.

ID	Question	Status	One-line answer
B17	Can one browser register as two Subscribers?	DOCUMENTED (not observed)	Possible in the SDK with a separate storageImplementation per client; platform acceptance unverified.
B18	Does connect_result differ between decline, no answer, and no registrations?	DOCUMENTED (not observed)	Documented: connect_result is connected or failed only, so all three cases read failed.
C1	Is the \$3/month Subscriber fee billed per Subscriber created or per Subscriber registered in the period?	NOT DETERMINABLE EMPIRICALLY	Not public: \$3.00/month per Subscriber is listed, but the billing basis is not stated. Confirm with Sales.
C2	How is the outbound CPS limit raised, and to what?	NOT DETERMINABLE EMPIRICALLY	Default 1 CPS; raise via Support, paid tiers \$15 to \$40/month listed. Confirm ceiling with Sales/Support.
C3	911 from browser softphones	NOT DETERMINABLE EMPIRICALLY	Not tested. E911 is per number with a static US address; WebRTC 911 is undocumented. Review with counsel.
C4	Can numbers ported from Telnyx carry both existing 10DLC campaigns and voice?	NOT DETERMINABLE EMPIRICALLY	Voice ports freely; 10DLC campaigns need re-registration or a CNP change. Confirm with Carrier Ops.

Method notes

- PSTN simulation: A to B calls are SignalWire-to-SignalWire between numbers owned by the test Space and may not traverse an external carrier. Every dial asserts the destination is owned by the Space and not on the emergency or N11 denylist (harness/config.py).
- Documentation sources: the public signalwire.com/docs site was blocked by the network policy (HTTP 403), so docs were read through the SignalWire knowledge server, which indexes the same public pages and returns their URLs. Each relied-on page is saved under evidence/docs/ with source and fetch time. Some SWML details come from the knowledge server's SWML reference rather than a public page; those are labeled in evidence/docs/swml/.
- SDK facts come from the published npm package @signalwire/js@4.0.0-rc.4 type declarations and implementation, read without executing it, with line citations in evidence/sdk/sdk_facts.md. The browser self-test then loaded that bundle in headless Chromium to confirm the exported identifiers.
- Every behavioral test is coded to run 3 times (harness/tests.py, RUNS = 3). A result is definitive only when all runs agree.

Part A: questions the customer asked

A1. Does calling one Subscriber ring all of its registered devices?

Status: DOCUMENTED (not observed)

Method: live test A1 in harness/tests.py, hypothesis: Calling one Subscriber rings every registered device. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/docs/extra/multi_device_and_reject.md, evidence/sdk/sdk_facts.md (Item 1, register)

Customer-facing answer: Documented yes. The Server SDK dial reference states that dialing a Subscriber's resource address "rings every live registration of a subscriber at once". The Browser SDK side only rings while a device is registered (register() marks the Subscriber online). Not yet observed with 3 devices.

A2. First-answer-wins: when one device answers, do the others stop ringing, and how fast?

Status: BLOCKED

Method: live test A2 in harness/tests.py, hypothesis: When one device answers, the other devices leave 'ringing'. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/sdk/sdk_facts.md (Item 2, CallStatus)

Customer-facing answer: Not answered by documentation. The SDK exposes per-call status\$ with values new, trying, ringing, connecting, connected, recovering, disconnecting, disconnected, failed, destroyed, so the harness can time the transition off ringing on the other devices, but neither the docs nor the SDK state what the losing devices see or how fast. Needs the live run.

A3. Does reject() on one device stop all devices on that Subscriber, and other Subscribers in the same parallel group?

Status: BLOCKED

Method: live test A3 in harness/tests.py, hypothesis: reject() on one device stops ringing on all devices of that Subscriber. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/docs/extra/multi_device_and_reject.md, evidence/docs/rest_facts.json (1_calling_rest_api)

Customer-facing answer: Not answered by documentation. The reject() reference says "the caller-side rings stop, no media is negotiated, and the call instance transitions to disconnected" for the rejecting device, but nothing documents the effect on other devices of the same Subscriber or on other parallel destinations. The backend workaround (calling.end on the anchor leg via REST) uses documented commands; its stop latency needs the live run.

A4. What does the inbound SWML webhook request contain?

Status: DOCUMENTED (not observed)

Method: live test A4 in harness/tests.py, hypothesis: The inbound SWML webhook request carries call.call_id, call.from and call.to. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/docs/swml/public_swml_calling_webhook_payload.md, evidence/docs/swml_facts.json (5)

Customer-facing answer: Documented: an HTTP POST with a JSON body whose top-level keys are call, vars, envs, params (params is {} on the first inbound fetch). call carries call_id, call_state, direction, from, to, headers[], node_id, project_id, segment_id, space_id, type (sip, phone, webrtc) and sip_data for SIP. Requests are signed with X-SignalWire-Signature. Reply with application/json or YAML. Exact live payload not yet captured.

A5. Can the browser dial an SWML webhook address with userVariables, and what caller identity does the webhook receive?

Status: DOCUMENTED (not observed)

Method: live test A5 in harness/tests.py, hypothesis: Browser dial() to an SWML webhook address delivers userVariables to the webhook under vars.userVariables. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/sdk/sdk_facts.md (Item 3), evidence/docs/swml_facts.json (5), evidence/docs/extra/multi_device_and_reject.md

Customer-facing answer: Partly documented. The v4 dial shape is confirmed from the published SDK types: client.dial('/private/<name>', { audio: true, video: false, userVariables: { to, crm_call_id } }); there is no to option, the destination is the first argument. Which webhook field carries userVariables, and which field identifies the calling Subscriber, are not documented for the SWML fetch: vars.userVariables appears only in SWAIG examples, and call.from is described only as "the number/URI that initiated this call". The Browser SDK guide shows a Subscriber from as an address like /private/alice. The forged-identity check needs the live run.

A6. connect serial fallback to a phone number after the Subscriber times out, and connect_result branching

Status: DOCUMENTED (not observed)

Method: live test A6 in harness/tests.py, hypothesis: connect.serial falls back to a phone number after the Subscriber times out. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/docs/swml/public_swml_connect.md, evidence/docs/swml_facts.json (1, 2)

Customer-facing answer: Documented: connect accepts serial (try in order), parallel or serial_parallel, with a per-destination timeout, so a Subscriber address followed by a phone number gives fallback. After connect, connect_result is connected or failed (and return_value equals it); connect.result or a switch on connect_result selects the voicemail branch. connect_failed_reason has no documented value list. Not yet observed.

A7. Exact REST command names and payloads for record start, pause, resume, stop, and hangup

Status: DOCUMENTED (not observed)

Method: live test A7 in harness/tests.py, hypothesis: calling.record / record.pause / record.resume / record.stop / calling.end succeed on the anchor leg. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/docs/rest/calling_api_call_commands.md, evidence/docs/rest_facts.json (1_calling_rest_api)

Customer-facing answer: Documented on POST /api/calling/calls with Basic auth: {"command":"calling.record","id":<call_id>,"params":{"control_id":..,"audio":{"format":"mp3","stereo":true,"direction":"both"}}}; calling.record.pause with {control_id, behavior} where behavior is skip (default) or silence; calling.record.resume and calling.record.stop with {control_id}; hangup is calling.end with {reason:"hangup"}. The exact nesting of audio under params comes from SDK-level docs and needs the live run to confirm.

A8. Stereo MP3 of both parties: format and channel mapping, inbound and outbound

Status: BLOCKED

Method: live test A8 in harness/tests.py, hypothesis: A stereo recording puts each party on its own channel. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/docs/swml_facts.json (3, gaps), evidence/selftest/audio/result.json

Customer-facing answer: Not answered by documentation. stereo: true with format: mp3 is a documented option, but which party lands on which channel is not documented. The FFT analyzer that answers this is built and passes its self-test on synthetic audio; it needs real recordings.

A9. Which status webhooks fire for connect, recording and call end, and where is the REST recording callback configured?

Status: DOCUMENTED (not observed)

Method: live test A9 in harness/tests.py, hypothesis: Connect, recording and call-end status webhooks are delivered. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/docs/swml_facts.json (1, 3), evidence/docs/rest/swml_record_call_status_callbacks.md

Customer-facing answer: Documented events: `connect.status_url` posts `calling.call.connect` (`connect_state` `connecting`, `connected`, `failed`, `disconnected`); `call.state_url` posts `calling.call.state` (`created`, `ringing`, `answered`, `ended` with `end_reason`); `recording.status_url` posts `calling.call.record` (`state` `recording`, `paused`, `finished`, `no_input`, `error`, with `url`, `recording_id`, `duration`, `size`). For a REST-started recording the callback goes in `params.status_url` of `calling.record` per SDK-level docs. Which location delivers in practice needs the live run.

A10. Which `call_id` must REST commands target (anchor rule)?

Status: BLOCKED

Method: live test A10 in `harness/tests.py`, hypothesis: REST recording commands succeed only on the anchor leg. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: `evidence/docs/rest_facts.json` (`1_calling_rest_api`, `gaps`)

Customer-facing answer: Not documented. No source searched states an anchor-leg rule for REST commands. The harness sends `calling.record` to both legs on inbound and outbound calls to settle it.

A11. Maximum devices per Subscriber

Status: BLOCKED

Method: live test A11 in `harness/tests.py`, hypothesis: Every registered device rings (5, 10, 20 devices). 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: `evidence/sdk/sdk_facts.md` (Item 7), `evidence/docs/extra/multi_device_and_reject.md`

Customer-facing answer: Not documented. No maximum devices per Subscriber appears in docs, and the SDK has no client-side device limit constant. Needs the 5, 10, 20 device run.

A12. Cost per call by product

Status: BLOCKED

Runs: none executed. Live test blocked (see Run outcome).

Evidence: `evidence/docs/rest/pricing.md`, `evidence/docs/rest/voice_logs_api.md`

Customer-facing answer: No calls were placed, so there is no per-call cost data. Published list rates (not observed charges): WebRTC \$0.003/min each way, local PSTN \$0.0066/min inbound and \$0.008/min outbound, call recording \$0.002/min. Per-call charges come from `GET /api/voice/logs`; its cost fields are not documented and must be read from a live response.

Part B: questions they are likely to ask

B1. No device registered for a Subscriber in the parallel list: fail fast or wait for timeout?

Status: BLOCKED

Method: live test B1 in `harness/tests.py`, hypothesis: With zero registrations, connect fails before the configured timeout. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: `evidence/docs/swml_facts.json` (1)

Customer-facing answer: Not documented. Whether connect fails fast or waits for timeout when a Subscriber has no registrations is not stated. Needs the live run.

B2. Parallel destination cap for connect.parallel

Status: BLOCKED

Method: live test B2 in harness/tests.py, hypothesis: connect.parallel rings all 22 Subscribers. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/docs/swml_facts.json (1, gaps)

Customer-facing answer: Not documented. No parallel destination limit appears in SWML docs or the SWML JSON schema (no maxItems). The reported 20-destination limit is not in any source searched; the only related figure is cXML <Dial> allowing up to 10 <Sip> nouns, which is a different product.

B3. Does an expiring SAT drop an active call?

Status: BLOCKED

Method: live test B3 in harness/tests.py, hypothesis: An active call survives SAT expiry. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/sdk/sdk_facts.md (Item 1)

Customer-facing answer: Not documented for active calls. The SDK validates expiry_at only at construction (Provided credentials have expired) and on reconnect; nothing in the docs or SDK says an established media session is torn down at token expiry. Needs the live run.

B4. Does automatic refresh work (backend refresh vs fingerprint + sat:refresh)?

Status: DOCUMENTED (not observed)

Method: live test B4 in harness/tests.py, hypothesis: The refresh() provider is called before expiry and registration continues. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/sdk/sdk_facts.md (Item 1),
evidence/docs/rest/fabric_subscribers_and_tokens.md

Customer-facing answer: Documented in the SDK: pass a provider with refresh() and expiry_at (ms); the SDK calls refresh() 5 s before expiry and retries up to 5 times with backoff, then emits TokenRefreshError on client.errors\$ and disconnects. With a token minted with scope: ["sat:refresh"] and the SDK's fingerprint, the SDK refreshes on its own through /api/fabric/subscriber/devices/token and /devices/refresh. Backend refresh uses POST /api/fabric/subscribers/tokens/refresh {refresh_token}. Not yet observed.

B5. Does deleting a Subscriber revoke active sessions and in-progress calls?

Status: BLOCKED

Method: live test B5 in harness/tests.py, hypothesis: Deleting a Subscriber ends its active registration. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/docs/rest_facts.json (2_fabric_subscribers)

Customer-facing answer: Not documented. DELETE /api/fabric/resources/subscribers/{id} returns 204; the effect on live registrations and in-progress calls is not stated in public docs. Needs the live run.

B6. Can custom data reach the browser on inbound calls?

Status: DOCUMENTED (not observed)

Method: live test B6 in harness/tests.py, hypothesis: connect headers or params reach the browser Call object. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/sdk/sdk_facts.md (Item 2), evidence/docs/swml_facts.json (1)

Customer-facing answer: Documented as not available through headers: the v4 inbound Call object has no headers or custom-data property, and SWML connect.headers is documented as having no effect on calls to phone numbers, with no statement about WebRTC destinations. The SDK maps userVariables from the inbound invite onto call.userVariables, so that is the only documented path for custom data. Whether SWML can populate it for a Subscriber destination needs the live run.

B7. What caller ID does the browser see on inbound, and can connect.from override it?

Status: BLOCKED

Method: live test B7 in harness/tests.py, hypothesis: The browser sees the original PSTN caller number, and connect.from overrides it. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/sdk/sdk_facts.md (Item 2), evidence/docs/extra/multi_device_and_reject.md, evidence/docs/swml_facts.json (8)

Customer-facing answer: Not documented for PSTN callers. The SDK maps from from the invite's caller_id_number and fromName from caller_id_name. The Browser SDK guide shows from as an address for Subscriber-to-Subscriber calls. Whether a PSTN caller appears as the original number or the Space number, and whether connect.from overrides it for a Resource Address, are not documented.

B8. What does the remote party hear during browser hold, and is it recorded?

Status: BLOCKED

Method: live test B8 in harness/tests.py, hypothesis: The remote party hears silence (no 440 Hz) during browser hold, and the recording shows it. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/sdk/sdk_facts.md (Item 4), evidence/docs/swml/public_callerid_and_hold.md

Customer-facing answer: Not documented for the Browser SDK. call.toggleHold() is the only hold control in v4. RELAY server-side hold docs say the remote party "hears hold music or silence", without saying which. Needs the recording FFT.

B9. Can a device on an active call receive a second inbound call?

Status: DOCUMENTED (not observed)

Method: live test B9 in harness/tests.py, hypothesis: A device on an active call is offered a second inbound call. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/docs/extra/multi_device_and_reject.md, evidence/sdk/sdk_facts.md (Item 2)

Customer-facing answer: Documented at the SDK level: incomingCalls\$ emits the full list of inbound calls and "two simultaneous callers land in the same emission"; the SDK does not queue or pick. Whether the platform offers a second call to a device already on a call needs the live run.

B10. Server-side transfer of a connected call to another Subscriber and to a phone number

Status: DOCUMENTED (not observed)

Method: live test B10 in harness/tests.py, hypothesis: calling.transfer moves a connected call to another Subscriber and to a phone number. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/docs/rest_facts.json (1_calling_rest_api), evidence/sdk/sdk_facts.md (Item 4)

Customer-facing answer: Documented: calling.transfer with dest (an SWML URL or inline document) on the call; SWML transfer also takes dest. Pointing dest at SWML that connects to another Subscriber address or a phone number is the documented pattern. The browser-side call.transfer({destination}) also exists in v4. What each party experiences needs the live run.

B11. Webhook authenticity: signature header and validation

Status: DOCUMENTED (not observed)

Method: live test B11 in harness/tests.py, hypothesis: Webhooks carry X-SignalWire-Signature that validates with the Signing Key. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/docs/swml/public_swml_webhook_security.md,
evidence/docs/swml/sdk_webhook_signature_validation.md,
evidence/selftest/signature/result.json

Customer-facing answer: Documented: X-SignalWire-Signature is hex HMAC-SHA1 and X-SignalWire-Sha256-Signature is hex HMAC-SHA256, both over url + raw_body, keyed with the project Signing Key (not the API token). The harness validator agrees with SignalWire's published Python RequestValidator and rejects a tampered body (local self-test). Not yet checked against a live webhook.

B12. Webhook reliability: retries and timeouts when the backend is slow or returns 500

Status: DOCUMENTED (not observed)

Method: live test B12 in harness/tests.py, hypothesis: A slow or failing SWML endpoint is retried. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/docs/swml/public_swml_webhook_reliability.md,
evidence/docs/swml/public_compat_webhooks_timeouts_retries.md

Customer-facing answer: Partly documented: SignalWire waits about 5 seconds for SWML, then uses fallback_request_url on timeout, 5xx or invalid SWML, and the call fails with http_retrieval_error if nothing usable returns. Retry counts for SWML fetches and SWML status callbacks are not documented (the 3-attempt status callback retry applies to cXML).

B13. Recording retention and deletion via API

Status: DOCUMENTED (not observed)

Method: live test B13 in harness/tests.py, hypothesis: A recording can be listed, fetched and deleted, and its URL stops serving. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/docs/rest/recordings_api.md

Customer-facing answer: Documented: GET /api/relay/rest/recordings, GET and DELETE /api/relay/rest/recordings/{id} (also the cXML Recordings endpoints). Retention: the cXML docs say recordings remain stored indefinitely until deleted. Whether the media URL stops serving after delete needs the live run.

B14. Outbound CPS: what happens when the default limit is exceeded?

Status: DOCUMENTED (not observed)

Method: live test B14 in harness/tests.py, hypothesis: Bursts above the default CPS are queued rather than rejected. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/docs/rest/platform_rate_limits.md

Customer-facing answer: Documented: the default is 1 call per second, and calls above it are queued (backlog of 10,000) rather than rejected; when the backlog is full SignalWire stops adding calls. The error returned at that point is not documented.

B15. Negotiated codecs on the WebRTC leg and the PSTN leg

Status: BLOCKED

Method: live test B15 in harness/tests.py, hypothesis: Codecs are visible in getStats(). 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/sdk/sdk_facts.md (Items 3, 6), evidence/docs/swml_facts.json (1)

Customer-facing answer: Not documented. The SDK offers preferredAudioCodecs on dial and exposes call.rtcPeerConnection for getStats(); it has no codec getter. SWML connect.codecs sets offered codecs. Negotiated codecs need the live run.

B16. Can one Subscriber hold multiple simultaneous connected calls across devices?

Status: BLOCKED

Method: live test B16 in harness/tests.py, hypothesis: One Subscriber can hold two simultaneous connected calls on two devices. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/docs/extra/multi_device_and_reject.md

Customer-facing answer: Not documented. No source states whether one Subscriber can hold two connected calls on two devices at once. Needs the live run.

B17. Can one browser register as two Subscribers?

Status: DOCUMENTED (not observed)

Method: live test B17 in harness/tests.py, hypothesis: One browser can register as two Subscribers. 3 runs.

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/sdk/sdk_facts.md (Item 5)

Customer-facing answer: Partly documented from the SDK source: multiple SignalWire instances can be created in one page, but sw:cached_credential, sw:client_bound, sw:preferences and the IndexedDB DPoP key are shared across instances. Pass a separate storageImplementation to each client to keep two Subscribers apart. Whether the platform accepts both registrations needs the live run.

B18. Does connect_result differ between decline, no answer, and no registrations?

Status: DOCUMENTED (not observed)

Runs: none executed. Live test blocked (see Run outcome).

Evidence: evidence/docs/swml_facts.json (1)

Customer-facing answer: Documented: connect_result has two values, connected and failed, so decline, no answer and no registrations all read failed. Any distinction would be in connect_failed_reason (no documented values) or in call_state_url end_reason (hangup, busy, no_answer, cancel, declined, error).

Part C: not determinable by testing

C1. Is the \$3/month Subscriber fee billed per Subscriber created or per Subscriber registered in the period?

Status: NOT DETERMINABLE EMPIRICALLY

Confirm with: Sales

Evidence: evidence/docs/rest/pricing.md

Customer-facing answer: Public pricing lists Subscriber at \$3.00/month and the Subscribers page says "\$3/user/month". No public source says whether this is charged per Subscriber created, per Subscriber existing, or per Subscriber that registered in the period, and no usage API documented exposes Subscriber line items. Confirm with Sales.

C2. How is the outbound CPS limit raised, and to what?

Status: NOT DETERMINABLE EMPIRICALLY

Confirm with: Sales / Support

Evidence: evidence/docs/rest/platform_rate_limits.md, evidence/docs/rest/pricing.md

Customer-facing answer: Docs: the default is 1 CPS with a 10,000-call backlog. Throughput is raised by contacting Support; the backlog by the Space Increase Request Form. The pricing page lists paid CPS tiers at \$15 to \$40 per month each with a 3-month commitment. The ceiling for this account must come from Sales or Support.

C3. 911 from browser softphones

Status: NOT DETERMINABLE EMPIRICALLY

Confirm with: Counsel and SignalWire Support (E911)

Evidence: evidence/docs/rest/e911.md

Customer-facing answer: Not tested by design. SignalWire E911 is configured per phone number with a registered US address (POST /api/relay/rest/addresses, then assigned to the number; status pending then active) at \$0.75/month. There is no dynamic location: PIDF-LO is not used and the docs say to update the address when a device moves. Nothing in the docs covers 911 placed from WebRTC Subscribers. For counsel: Kari's Law requires direct 911 dialing without a prefix and on-site notification for multi-line telephone systems, and RAY BAUM's Act Section 506 requires a dispatchable location, including for non-fixed devices. Neither law is addressed in the SignalWire docs searched.

C4. Can numbers ported from Telnyx carry both existing 10DLC campaigns and voice?

Status: NOT DETERMINABLE EMPIRICALLY

Confirm with: Carrier Ops

Evidence: evidence/docs/rest/porting_and_10dlc.md

Customer-facing answer: Docs: voice and messaging both work on ported numbers, but 10DLC campaigns do not transfer automatically. Each messaging provider is its own CSP registration, so a customer migrating from another provider re-registers Brand and Campaign with SignalWire, or, if they are their own CSP, selects SignalWire as the upstream CNP. The old registration stays in place until deactivated. Port-in is free and takes about a week. Confirm with Carrier Ops whether an existing campaign can be moved by CNP change without re-vetting.